Guía del alumno

Módulo 1, Importar y unir datos

Ruta guiada

Análisis Avanzado de Datos con Python

Cómo usar esta guía

Esta guía acompaña al Lab 01 guiado y sigue sus dos partes en el mismo orden y con los mismos nombres del cuaderno. La Parte 1, En clase, tiene cinco pasos, de Paso 1 de 5 a Paso 5 de 5, y se hace en clase. La Parte 2, Para estudiar, tiene cinco estudios, de Estudio 1 de 5 a Estudio 5 de 5, y se hace después de clase con esta guía, una vez corrida la Parte 1. Cada paso trae el código y la respuesta tal como aparecen en Colab, y una explicación en palabras simples.

No hay que escribir código. Lo que cambia se cambia en las listas desplegables y en la casilla, y después se vuelve a correr la celda. Los datos se leen desde la carpeta de datos del curso en GitHub, así que hace falta estar conectado a internet.

Lo que vas a poder hacer al terminar

- Abrir una tabla y mirar su tamaño, sus columnas y un total.
- Filtrar las filas que interesan y resumir la tabla por grupos.
- Reconocer que los datos pueden llegar de un CSV, un Excel, un servicio web o una base de datos.
- Unir dos tablas y comprobar que no se perdieron ni se duplicaron filas.

Todas las respuestas de esta guía vienen de una corrida del cuaderno. Los datos del curso son sintéticos, inventados para aprender, y no se citan como datos del sector.

Parte 1 · En clase

Cinco pasos, uno detrás del otro, que se hacen en clase en el mismo orden del cuaderno.

Paso 1 de 5 · ¿Qué centrales tenemos?

Todo análisis parte por abrir la tabla y mirarla. Esta es la tabla de las 20 centrales del curso.

Paso 1 de 5

```
# Paso 1
# La carpeta de datos del curso, en el repositorio.
DATOS = "https://raw.githubusercontent.com/RodrigoMSB/curso-python-minergia/main/datos/"

import pandas as pd

# read_csv lee un archivo separado por comas y lo convierte en una tabla.
centrales = pd.read_csv(DATOS + "centrales.csv")
centrales
```

```
                        central tecnologia         region  potencia_mw  \
0         Central Río Manso Alto      hidro         Biobío          420
1            Central Salto Verde      hidro      Los Lagos          310
2           Central Aguas Claras      hidro         Biobío          180
3              Central Vega Azul      hidro      Los Lagos           95
4            Central Tres Saltos      hidro         Biobío          260
5        Parque Solar Pampa Alta      solar    Antofagasta          230
6        Parque Solar Llano Seco      solar        Atacama          180
7      Parque Solar Sol Naciente      solar    Antofagasta          145
8    Parque Solar Quebrada Honda      solar        Atacama           95
9          Parque Solar Altiplano      solar    Antofagasta          310
10            Eólica Cerro Negro     eolica       Coquimbo          160
11          Eólica Punta Ventosa     eolica       Coquimbo          120
12              Eólica Loma Fría     eolica     Valparaíso           85
13          Eólica Campo Abierto     eolica       Coquimbo          200
14    Termoeléctrica Bahía Norte        gas     Valparaíso          375
15     Termoeléctrica Puerto Sur        gas         Biobío          290
16  Termoeléctrica Valle Central        gas  Metropolitana          210
17    Carboeléctrica Costa Brava     carbon         Biobío          480
18      Carboeléctrica Roca Gris     carbon    Antofagasta          350
19    Diésel Respaldo Cordillera     diesel  Metropolitana           45

    anio_inicio
0          2004
1          1998
2          2011
3          2016
4          1995
5          2019
6          2020
7          2021
8          2022
9          2023
10         2017
11         2018
12         2020
13         2021
14         2008
15         2012
16         2006
17         2001
18         1999
19         2014
```

`DATOS` guarda la dirección de la carpeta de datos del curso en GitHub. Así cada archivo se pide sumando su nombre a esa dirección.

`import pandas as pd` trae pandas, la herramienta de Python que trabaja con tablas. `pd.read_csv` lee un archivo CSV, que es un archivo de texto con los valores separados por comas, y lo convierte en una tabla. La tabla queda guardada con el nombre `centrales`.

La tabla es más ancha que la hoja, así que en esta guía la última columna aparece más abajo. En Colab se ve completa. Los números de la izquierda numeran las filas y empiezan en cero.

Paso 2 de 5 · Tu turno, filtra por tecnología

Un filtro deja solo las filas que interesan. Acá, las centrales de una tecnología.

Paso 2 de 5

```
# Paso 2
# La tabla deja solo las centrales de la tecnología elegida.
tecnologia = "hidro" #@param ["hidro", "solar", "eolica", "gas", "carbon", "diesel"]

filtradas = centrales[centrales["tecnologia"] == tecnologia]
print(f"✓ Hay {len(filtradas)} centrales de {tecnologia}")
filtradas
```

```
✓ Hay 5 centrales de hidro
                central tecnologia     region  potencia_mw  anio_inicio
0  Central Río Manso Alto      hidro     Biobío          420         2004
1     Central Salto Verde      hidro  Los Lagos          310         1998
2    Central Aguas Claras      hidro     Biobío          180         2011
3       Central Vega Azul      hidro  Los Lagos           95         2016
4     Central Tres Saltos      hidro     Biobío          260         1995
```

La tecnología se elige en la lista desplegable. La celda compara la columna `tecnologia` con lo que elegiste y se queda con las filas donde coinciden. Viene marcada la hidro, y hay 5 centrales.

Elige otra tecnología y vuelve a correr la celda. Las tecnologías van sin tilde, `eolica` y `diesel`, porque así están escritas en los datos.

Paso 3 de 5 · ¿Cómo llega la generación de cada hora?

Otro CSV, mucho más grande que el de las centrales.

Paso 3 de 5

```
# Paso 3
# parse_dates avisa que la columna fecha trae fechas y no texto.
generacion = pd.read_csv(DATOS + "generacion.csv", parse_dates=["fecha"])

print("Filas y columnas", generacion.shape)
print("Primer día      ", generacion["fecha"].min().date())
print("Último día      ", generacion["fecha"].max().date())
generacion.head()
```

```
Filas y columnas (175680, 4)
Primer día       2024-01-01
Último día       2024-12-31
       fecha  hora                central     mwh
0 2024-01-01     0  Central Río Manso Alto  163.69
1 2024-01-01     1  Central Río Manso Alto  179.81
2 2024-01-01     2  Central Río Manso Alto  173.77
3 2024-01-01     3  Central Río Manso Alto  170.39
4 2024-01-01     4  Central Río Manso Alto  169.84
```

El archivo trae la generación de cada central en cada hora de 2024. Son 175680 filas y 4 columnas, la fecha, la hora, la central y los MWh generados. Va del 1 de enero al 31 de diciembre de 2024.

`parse_dates` le avisa a pandas que la columna `fecha` trae fechas y no texto. Así se le puede pedir el primer día y el último. `head` muestra solo las primeras filas, porque mostrar todas no sirve de nada.

Paso 4 de 5 · Tu turno, une generación con fichas

Muchas preguntas necesitan dos tablas a la vez. Para saber cuánto generó cada tecnología, hay que juntar la generación con la ficha de cada central, que es la que dice su tecnología. Juntar dos tablas se llama unir.

Paso 4 de 5

```
# Paso 4
# inner deja solo lo que está en las dos tablas.
# left conserva toda la generación.
como_unir = "inner" #@param ["inner", "left"]

# Un día de generación y una tabla de fichas a la que le faltan
# tres centrales.
un_dia = generacion[generacion["fecha"] == "2024-06-15"]
sin_ficha = ["Eólica Loma Fría", "Central Vega Azul", "Diésel Respaldo Cordillera"]
fichas = centrales[~centrales["central"].isin(sin_ficha)]

unido = pd.merge(un_dia, fichas, on="central", how=como_unir)
print("Generación real del día", round(un_dia["mwh"].sum(), 1), "MWh")
print("Generación unida       ", round(unido["mwh"].sum(), 1), "MWh")
print()
# Las centrales sin ficha quedan agrupadas como "sin ficha".
unido_con_ficha = unido.fillna({"tecnologia": "sin ficha"})
por_tecnologia = unido_con_ficha.groupby("tecnologia")["mwh"].sum().round(1)
print(por_tecnologia.sort_values(ascending=False))
```

```
Generación real del día 52650.0 MWh
Generación unida        50395.9 MWh

tecnologia
hidro     18782.7
gas       13999.4
carbon    12746.9
eolica     3476.7
solar      1390.1
Name: mwh, dtype: float64
```

La celda toma un día de generación y una tabla de fichas a la que le faltan tres centrales. Después las une por el nombre de la central, con la forma que elijas en la lista.

Con `inner` la unión deja solo las centrales que están en las dos tablas. Por eso la generación unida, 50395.9 MWh, es menor que la real, 52650.0. Las tres centrales sin ficha se perdieron sin ningún aviso.

Elige `left` y vuelve a correr la celda. Ahora la unión conserva toda la generación, y la generación unida es igual a la real. Las centrales sin ficha quedan agrupadas como *sin ficha* en el resumen por tecnología.

> **ANTES Y DESPUÉS DE UNIR**
>
> Inner deja fuera, sin avisar, las centrales que no tienen ficha. Left conserva toda la generación y deja vacía la ficha que falta. Por eso, antes y después de unir se cuentan las filas y se compara el total.

Paso 5 de 5 · Tu turno, ¿qué pasa con fichas repetidas?

El otro error común al unir es el contrario. Si la tabla de fichas trae una central dos veces, la unión repite sus filas y el total crece.

Paso 5 de 5

```
# Paso 5
# Marca la casilla para dejar una sola ficha por central.
quitar_repetidas = False #@param {type:"boolean"}

# Una tabla de fichas como llega en la práctica, con dos
# centrales repetidas.
fichas_repetidas = pd.concat([fichas, fichas.head(2)], ignore_index=True)
if quitar_repetidas:
    fichas_repetidas = fichas_repetidas.drop_duplicates(subset="central")

unido = pd.merge(un_dia, fichas_repetidas, on="central", how="left")
real = round(un_dia["mwh"].sum(), 1)
unida = round(unido["mwh"].sum(), 1)
print("Filas antes de unir", un_dia.shape[0], "y después", unido.shape[0])
print("Generación real", real, "MWh y generación unida", unida, "MWh")
if unida == real:
    print("✓ La generación unida calza con la real")
```

```
Filas antes de unir 480 y después 528
Generación real 52650.0 MWh y generación unida 64298.7 MWh
```

La celda arma una tabla de fichas con dos centrales repetidas y la une con el día de generación. Con la casilla sin marcar, las filas pasan de 480 a 528 y la generación unida, 64298.7 MWh, queda mayor que la real, 52650.0. La unión inventó generación que no existe.

Marca la casilla `quitar_repetidas` y vuelve a correr la celda. Ahora la celda deja una sola ficha por central antes de unir, las filas vuelven a ser 480 y aparece la línea `✓ La generación unida calza con la real`.

Las dos trampas de unir, perder filas y duplicarlas, se detectan igual. Se cuentan las filas y se compara el total antes y después.

Parte 2 · Para estudiar

Estos pasos se hacen después de clase, con esta guía. Antes de empezar hay que correr toda la Parte 1 del cuaderno, porque los estudios usan sus tablas.

Estudio 1 de 5 · ¿Qué tamaño tiene la tabla?

Antes de calcular nada conviene hacerle a la tabla tres preguntas. Cuántas filas y columnas tiene, cómo se llaman las columnas y cuánto suma la cifra principal.

Estudio 1 de 5

```
# Estudio 1
# Tres preguntas que uno le hace a cualquier tabla apenas la abre.
print("Filas y columnas", centrales.shape)
print("Columnas        ", list(centrales.columns))
print("Potencia total  ", centrales["potencia_mw"].sum(), "MW")
```

```
Filas y columnas (20, 5)
Columnas         ['central', 'tecnologia', 'region', 'potencia_mw', 'anio_inicio']
Potencia total   4540 MW
```

La tabla tiene 20 filas y 5 columnas, una central por fila. Las columnas son el nombre, la tecnología, la región, la potencia en MW y el año en que empezó a operar.

La potencia total de las 20 centrales es 4540 MW. Ese total sirve para comprobar que un cálculo no perdió ni duplicó filas, como en los Pasos 4 y 5.

Estudio 2 de 5 · Tu turno, resume la potencia

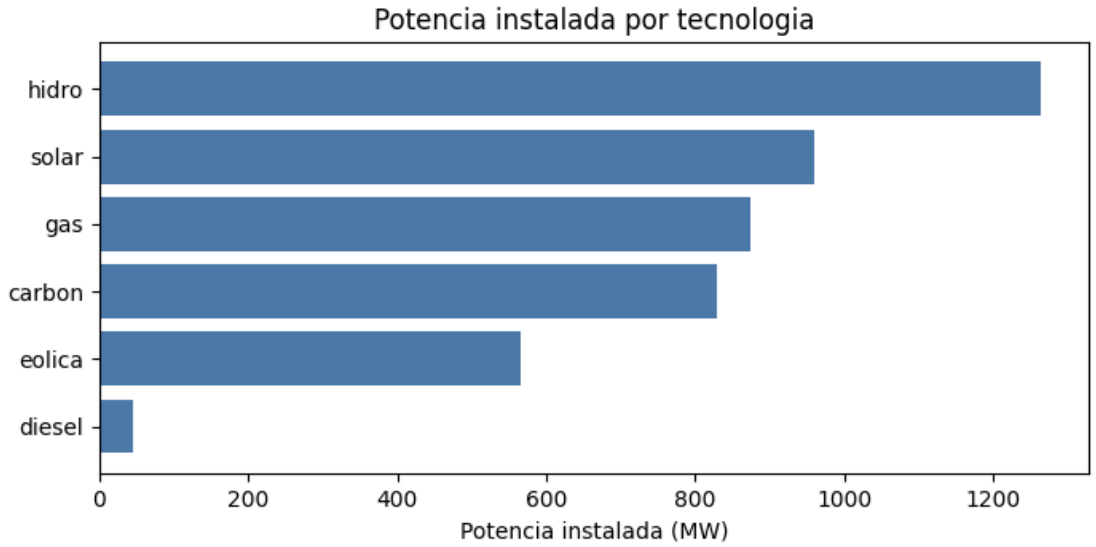
Un resumen junta las filas que comparten un valor y suma lo de cada grupo. Es lo mismo que hace una tabla dinámica en una planilla.

Estudio 2 de 5

```
# Estudio 2
# Suma la potencia de las centrales que comparten el valor elegido.
agrupar_por = "tecnologia" #@param ["tecnologia", "region"]

import matplotlib.pyplot as plt
resumen = centrales.groupby(agrupar_por)["potencia_mw"].sum().sort_values()

fig, ax = plt.subplots(figsize=(8, 3.5))
ax.barh(resumen.index, resumen.values, color="#4C78A8")
ax.set_title(f"Potencia instalada por {agrupar_por}")
ax.set_xlabel("Potencia instalada (MW)")
plt.show()
```



La lista desplegable elige cómo agrupar, por tecnología o por región. La celda suma la potencia de cada grupo, la ordena y dibuja un gráfico de barras. La barra más larga queda arriba.

Elige región y vuelve a correr la celda para ver el mismo total repartido de otra forma.

OJO CON ESTA CIFRA

Este parque de veinte centrales es inventado para el curso y no reproduce la matriz chilena. En él la solar es segunda en potencia instalada y quinta en energía generada, porque de noche no genera. Esa diferencia entre potencia y energía es justo lo que este bloque quiere mostrar.

Los datos del curso son sintéticos. Estas cifras no sirven para hablar del sistema eléctrico real.

Estudio 3 de 5 · ¿Y si los datos vienen en Excel?

Muchos datos llegan en una planilla Excel, y una planilla puede tener varias hojas.

Estudio 3 de 5

```
# Estudio 3
# Con sheet_name=None se traen todas las hojas del Excel de una vez.
hojas = pd.read_excel(DATOS + "centrales.xlsx", sheet_name=None)

print("Hojas del archivo", list(hojas.keys()))
hojas["notas"]
```

```
Hojas del archivo ['centrales', 'notas']
                                            nota
0    Potencias declaradas al 31 de diciembre de 2024
1  Las centrales de pasada se informan con su pot...
```

`pd.read_excel` lee el Excel. Con `sheet_name=None` trae todas las hojas de una vez. Este archivo tiene dos, `centrales` y `notas`.

La hoja `notas` trae aclaraciones sobre los datos, por ejemplo la fecha de las potencias. Conviene leerla siempre, porque explica cómo se armó la tabla.

Estudio 4 de 5 · ¿Y si vienen de un servicio web?

Hay datos que no vienen en un archivo sino que se le piden a un servicio en internet. Ese servicio responde con un texto armado en un formato llamado JSON.

Estudio 4 de 5

```
# Estudio 4
# Así se lee un servicio web de verdad. Aquí la dirección apunta a un archivo del curso.
import requests
respuesta = requests.get(DATOS + "precios_nudo.json").json()

print("Llaves de la respuesta", list(respuesta.keys()))

# json_normalize convierte la lista de registros en una tabla.
pd.json_normalize(respuesta["datos"]).head()
```

```
Llaves de la respuesta ['metadata', 'datos']
        fecha  hora       region  precio_usd_mwh
0  2024-01-01     0  Antofagasta           47.97
1  2024-01-01     1  Antofagasta           40.81
2  2024-01-01     2  Antofagasta           49.35
3  2024-01-01     3  Antofagasta           45.50
4  2024-01-01     4  Antofagasta           40.00
```

`requests.get` pide la respuesta a una dirección, y `.json()` la convierte en algo que Python entiende. Así se lee un servicio web de verdad. Aquí la dirección apunta a un archivo del curso.

La respuesta trae dos partes, `metadata` con la descripción y `datos` con los registros.

`pd.json_normalize` convierte la lista de registros en una tabla, una fila por fecha, hora y región, con el precio en dólares por MWh.

Estudio 5 de 5 · ¿Y si vienen de una base de datos?

En una institución, los datos grandes suelen estar en una base de datos. A la base se le hacen preguntas en un lenguaje llamado SQL.

Estudio 5 de 5

```
# Estudio 5
# Se baja el archivo de la base a Colab y se abre una conexión con él.
import sqlite3, urllib.request
urllib.request.urlretrieve(DATOS + "demanda.db", "demanda.db")
conexion = sqlite3.connect("demanda.db")

# El filtro se escribe en SQL y lo resuelve la base, así viaja mucho menos dato.
consulta = """
    SELECT region, SUM(mwh) AS demanda_total
    FROM demanda
    GROUP BY region
    ORDER BY demanda_total DESC
"""
pd.read_sql(consulta, conexion)
```

```
          region  demanda_total
0  Metropolitana     7779657.53
1     Valparaíso     1825770.20
2         Biobío     1632753.33
3      Los Lagos      864564.87
4       Coquimbo      816624.13
5    Antofagasta      672440.52
6        Atacama      307486.00
```

La celda baja el archivo de la base a Colab y se conecta a él. La pregunta en SQL pide la demanda total de cada región, de mayor a menor. La base hace la cuenta y entrega solo el resultado, siete filas, una por región. La Metropolitana queda primera.

No hace falta saber SQL para la ruta guiada. Lo que importa es ver que el resultado llega como una tabla, igual que desde un CSV o un Excel.

Para repasar

Referencia rápida

Lo que se escribe	Para qué sirve
`pd.read_csv(...)`	Lee un archivo CSV y lo convierte en tabla
`pd.read_excel(...)`	Lee una planilla Excel
`requests.get(...).json()`	Pide datos a un servicio web
`pd.read_sql(consulta, conexion)`	Le hace una pregunta a una base de datos
`tabla.shape`	Cuántas filas y columnas tiene la tabla
`tabla.head()`	Las primeras filas
`tabla[tabla["col"] == valor]`	Deja solo las filas que cumplen la condición
`groupby(...).sum()`	Junta las filas por grupo y suma cada uno
`pd.merge(a, b, on="central")`	Une dos tablas por una columna en común
`inner` · `left`	Solo lo que está en las dos, o todo lo de la primera

Si algo sale mal

Lo que aparece en pantalla	Qué pasó
`NameError: name 'centrales' is not defined`	Corriste un paso sin correr antes el Paso 1, que es el que lee la tabla. Corre los pasos en orden.
`NameError: name 'generacion' is not defined`	El Paso 4 usa la generación que lee el Paso 3. Corre el Paso 3 antes.
`URLError` o `HTTPError`	Colab no pudo bajar el archivo. Revisa la conexión a internet y vuelve a correr el paso.
La respuesta no cambió después de cambiar la lista	Cambiar la lista no corre la celda. Hay que volver a apretar el botón de play.

Tres preguntas de repaso

1. Para qué sirve mirar el total antes de unir dos tablas

Para comparar con el total después de unir. Si bajó, se perdieron filas. Si subió, se duplicaron.

2. Qué diferencia hay entre inner y left

Inner deja solo las filas que están en las dos tablas. Left conserva todas las filas de la primera, aunque no encuentren pareja.

3. Por qué crece el total cuando una ficha viene repetida

Porque cada fila de generación de esa central se une con las dos fichas y queda dos veces.

El Lab 01 guiado en una plana

Para imprimir y tener al lado durante el curso

Los pasos de las dos partes

N.	La pregunta	La idea
Parte 1, en clase		
1	¿Qué centrales tenemos?	Leer una tabla
2	Tu turno, filtra por tecnología	Quedarse con algunas filas
3	¿Cómo llega la generación de cada hora?	Un CSV grande, con fechas
4	Tu turno, une generación con fichas	Inner pierde, left conserva
5	Tu turno, ¿qué pasa con fichas repetidas?	Una ficha repetida duplica
Parte 2, para estudiar		
1	¿Qué tamaño tiene la tabla?	Filas, columnas y un total
2	Tu turno, resume la potencia	Sumar por grupo
3	¿Y si los datos vienen en Excel?	Una planilla con varias hojas
4	¿Y si vienen de un servicio web?	Una respuesta en JSON
5	¿Y si vienen de una base de datos?	Una pregunta en SQL

La regla de oro de unir tablas

Antes de unir	Cuenta las filas y anota el total
Después de unir	Vuelve a contar y compara
Si el total bajó	Se perdieron filas, revisa si conviene left
Si el total subió	Hay fichas repetidas, quita las repetidas antes de unir

Las otras formas de unir y el trabajo con archivos muy grandes están en la ruta completa de este módulo.